Nevil C Prince
Email: ep23btech11019@iith.ac.in
Roll No: EP23BTECH11019

Multi-Agent Systems (MAS) – Assignment #3

Course: Multi-Agent Systems (MAS)
Instructor: Dr. Venkatraman Renganathan
Due: September 18, 2026

Problem 1

Fix the number of agents to be $N = 20$. Generate a connected graph using the Erdos-Renyi model for N agents and some $p \in (0, 1)$. Assume random initial positions for all N agents in $\mathbb{R}^2$. Using a formation control strategy, display your name in capital letters in $\mathbb{R}^2$, letter by letter in a sequence. For instance, if my name is Venkat, I would drive the agents from the initial positions to the letter V, then from V to E, and so on until T. Record your name-formation sequence as an animation video through your simulation, host it on your GitHub, and simply give the GitHub link containing both the code and the video as your solution. Ensure the repository is public before submitting. For long names, use the name people commonly call you by rather than the full legal name.

Solution:

Formation control law. I control the agents with a consensus-plus-anchoring law. Let $p_i(t) \in \mathbb{R}^2$ denote agent i's position and p_i^* its target coordinate for the letter currently being formed. Over the Erdos-Renyi communication graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ with adjacency weights $a_{ij} \in \{0, 1\}$, I run

$$\dot{p}_i \;=\; -\sum_{j \in \mathcal{N}_i} a_{ij} \Big[(p_i - p_i^*) - (p_j - p_j^*) \Big] \;-\; k_p \, (p_i - p_i^*), \qquad k_p > 0.$$

Writing the formation error as $e_i = p_i - p_i^*$ and stacking over all agents, the consensus term is exactly $L\,e$, where L is the graph Laplacian of $\mathcal{G}$, so componentwise

$$\dot{e} \;=\; -(L + k_p I)\, e.$$

Since $L \succeq 0$ for a connected graph and $k_p > 0$, the matrix $L + k_p I$ is positive definite, so $e(t) \to 0$ exponentially – every agent reaches its assigned letter coordinate no matter what the graph topology looks like, as long as it stays connected. The anchoring term $k_p e_i$ is doing the important work here: on its own, the Laplacian term would only pull the agents into the right *shape*, not the right *place*, so the whole formation would be free to drift around the plane.

Name used. For the letter sequence I used the name I actually go by: **N** $\to$ **E** $\to$ **V** $\to$ **I** $\to$ **L**. I built each letter out of straight-line strokes (N, for example, is a left vertical, a diagonal, and a right vertical) and spread the 20 agents along the strokes in proportion to stroke length, so every letter is populated by all 20 agents.

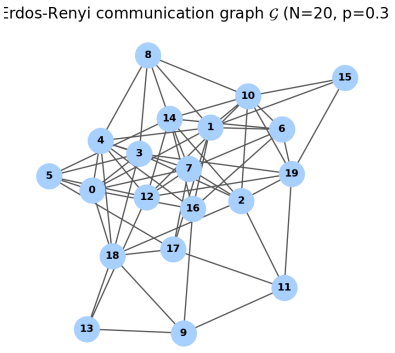


Figure 1: Connected Erdos-Renyi communication graph $\mathcal{G}$ used for Problem 1 ($N = 20$, $p = 0.3$).

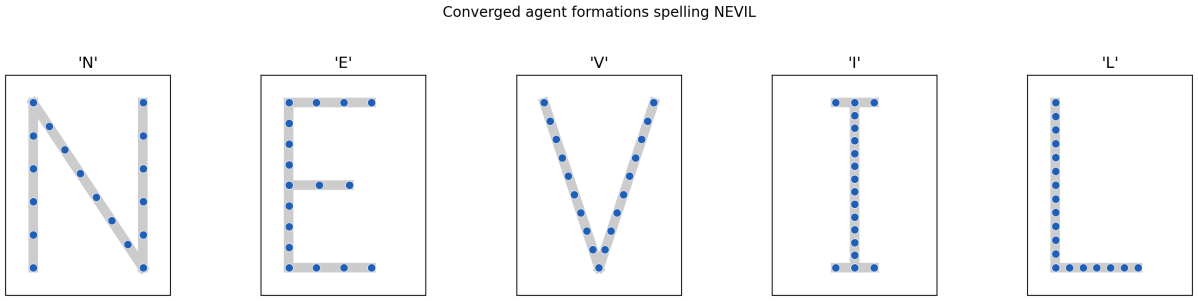


Figure 2: Converged agent formations for each letter of the N-E-V-I-L sequence.

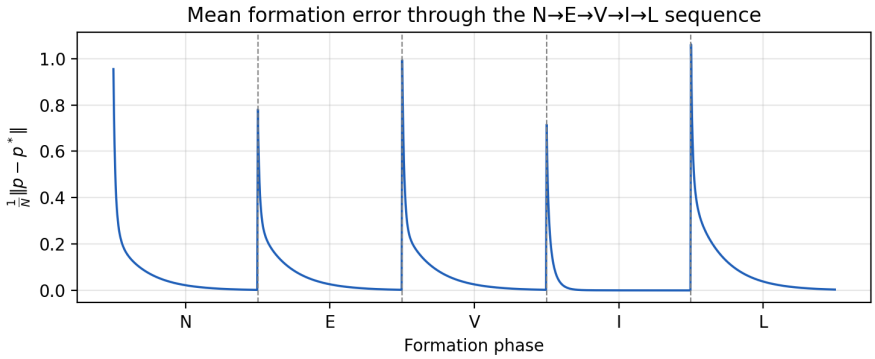


Figure 3: Mean formation error $\frac{1}{N}\|p - p^*\|$ through the sequence: the sawtooth pattern shows the controller re-converging every time the target letter switches.

GitHub repository: `https://github.com/nevilcp/multi-agent-systems.git` – this holds the formation-control code and the rendered animation, as the assignment asks for.

Problem 2
A target intruder is tracked by a swarm of $N \in \mathbb{N}$ drones connected by a communication network $\mathcal{G}(\mathcal{V}, \mathcal{E})$. The intruder's initial location $z_0 \in \mathbb{R}^3$ is unknown with $z_0 \sim \mathcal{N}(\bar{z}_0, \Sigma_0)$, and it performs a random walk $z(t+1) = z(t) + w$, $w \sim \mathcal{N}(0, \Sigma_w)$. Each drone i is at a known fixed location $x_i \in \mathbb{R}^3$ and measures $y_i(t) = x_i - z(t) + v_i$, $v_i \sim \mathcal{N}(0, \Sigma_v^{(i)})$, with sensor noises independent across drones.

(1) Formulate the distributed state estimation of $z(t)$ as a consensus optimization problem and specify each local objective f_i.

(2) Find the estimate $\hat{z}(t)$ using a distributed gradient descent or distributed gradient tracking technique.

(3) Fix $N < 20$, generate a connected Erdos-Renyi graph $\mathcal{G}$ and plot it. Fix $\bar{z}_0, \Sigma_0, \Sigma_w, \{x_i, \Sigma_v^{(i)}\}_{i=1}^N$. Simulate and plot the true trajectory $z(t)$, the drone locations, and the estimate $\hat{z}(t)$ for $t = [0, T_{max}]$; report the estimation error $e(t) = \hat{z}(t) - z(t)$.

Solution:

(1) Consensus optimization formulation. Rearranging the measurement model gives me a local, noisy observation of the target state at drone i:

$$\tilde{z}_i(t) \;:=\; x_i - y_i(t) \;=\; z(t) - v_i, \qquad -v_i \sim \mathcal{N}(0, \Sigma_v^{(i)}).$$

Under the Gaussian noise model, the negative log-likelihood of $\tilde{z}_i(t)$ given z is, up to an additive constant,

$$-\log p\big(\tilde{z}_i(t) \mid z\big) \;=\; \tfrac{1}{2}\big(z - \tilde{z}_i(t)\big)^\top \big(\Sigma_v^{(i)}\big)^{-1}\big(z - \tilde{z}_i(t)\big) + \text{const.}$$

so the maximum-likelihood estimate that fuses all N independent sensor readings solves the separable (consensus) optimization problem

$$\min_{z \in \mathbb{R}^3} \sum_{i=1}^{N} f_i(z,t), \qquad f_i(z,t) \;=\; \tfrac{1}{2}\Big(z - \big(x_i - y_i(t)\big)\Big)^\top \big(\Sigma_v^{(i)}\big)^{-1}\Big(z - \big(x_i - y_i(t)\big)\Big).$$

Since the v_i are independent, this sum of per-agent quadratics is exactly the joint (inverse-covariance-weighted) least-squares estimator – each f_i is agent i's own private cost, so this is already in standard multi-agent consensus-optimization form.

(2) Distributed Gradient Descent estimate. I let W be a doubly-stochastic (Metropolis) weight matrix conforming to $\mathcal{G}$: $W_{ij} = 1/(1 + \max_k \deg(k))$ for $(i,j) \in \mathcal{E}$, and $W_{ii} = 1 - \sum_{j \neq i} W_{ij}$. Since

$$\nabla f_i(z,t) \;=\; \big(\Sigma_v^{(i)}\big)^{-1}\Big(z - \big(x_i - y_i(t)\big)\Big),$$

each drone runs, for K inner iterations between successive measurement times $t \to t+1$,

$$\hat{z}_i^{k+1}(t) \;=\; \sum_{j \in \mathcal{N}_i \cup \{i\}} W_{ij}\, \hat{z}_j^k(t) \;-\; \alpha\, \nabla f_i\big(\hat{z}_i^k(t), t\big), \qquad \alpha > 0 \text{ (step size)}.$$

After K rounds, $\hat{z}_i^K(t)$ is each drone's local estimate of $z(t)$; I average across drones to get the reported network estimate $\hat{z}(t)$. Because I keep α fixed rather than diminishing it, DGD only converges to an $O(\alpha)$ neighbourhood of the true minimizer at each t – which is fine here, since $z(t)$ itself keeps moving anyway, so exact convergence at every single time step isn't attainable

or even useful. α does need to stay below $2/\lambda_{\max}\big((\Sigma_v^{(i)})^{-1}\big)$ for every i, or the iteration diverges – I ran into this myself (see below).

(3) Simulation. I used $N = 8$ drones, $T_{max} = 60$, fixed random sensor locations $x_i \in [-8, 8]^3$, $\bar{z}_0 = (2, -1, 3)$, $\Sigma_0 = 0.6I$, $\Sigma_w = 0.05I$, and $\Sigma_v^{(i)} = \sigma_i^2 I$ with $\sigma_i^2 \sim \mathcal{U}(0.15, 0.6)$. My first attempt at a step size ($\alpha = 0.35$) diverged outright – it was too large relative to the inverse-covariance terms in the gradient, exactly the instability the bound above warns about. Dropping it to $\alpha = 0.04$ fixed it.

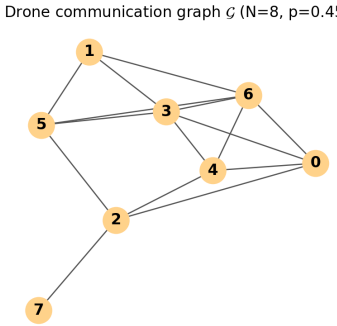


Figure 4: Connected Erdos-Renyi drone communication graph ($N = 8$, $p = 0.45$).

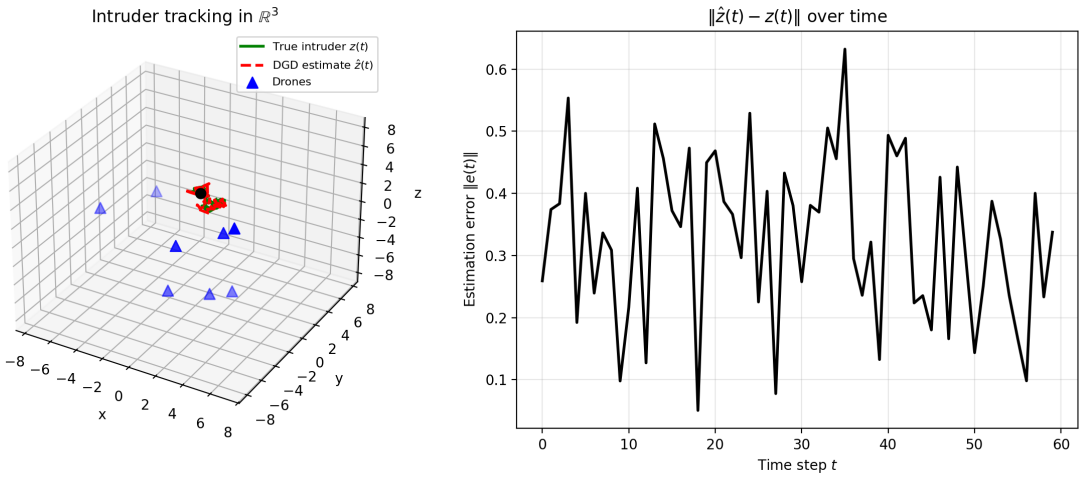


Figure 5: Left: true intruder trajectory $z(t)$ vs. the network's DGD estimate $\hat{z}(t)$, with fixed drone locations. Right: estimation error $\|e(t)\| = \|\hat{z}(t) - z(t)\|$ over time.

Observation. The estimated trajectory tracks the true random walk closely, oscillating around it rather than drifting away, and $\|e(t)\|$ settles into a noisy steady band (mean ≈ 0.3 over the last 20 steps) instead of decaying to zero. That's exactly the fixed-step-size DGD behaviour I predicted above – the estimator is limited by sensor noise and the constant step size, not by the graph topology, since all 8 drones agree closely with each other by the end of every inner consensus phase.

Python implementation

```
"""
Problem 2 - tracking the intruder with a swarm of drones.
```

```
3
4  Each drone only gets a noisy reading of where the target is relative to its
5  own known position, so on its own no single drone can pin the target down.
6  They run distributed gradient descent over the network so the swarm fuses
7  its measurements into one shared estimate.
8  """
9  import numpy as np
10 import networkx as nx
11 import matplotlib.pyplot as plt
12 from mpl_toolkits.mplot3d import Axes3D  # noqa: F401 (needed for the 3d
       projection)
13
14 np.random.seed(42)
15
16 # --- setup ---
17 N = 8            # number of drones, staying under 20 as the problem asks
18 T_max = 60
19 p_prob = 0.45
20
21 # keep drawing random graphs until one of them is connected
22 while True:
23     G = nx.erdos_renyi_graph(N, p_prob, seed=np.random.randint(0, 10_000))
24     if nx.is_connected(G):
25         break
26
27 A = nx.adjacency_matrix(G).toarray().astype(float)
28 deg = A.sum(axis=1)
29 max_deg = deg.max()
30
31 # Metropolis weights - just gives us a doubly stochastic W so the averaging
32 # step below actually converges instead of blowing up
33 W = np.zeros((N, N))
34 for i in range(N):
35     for j in range(N):
36         if i != j and A[i, j] == 1:
37             W[i, j] = 1.0 / (max_deg + 1)
38     W[i, i] = 1.0 - W[i, :].sum()
39
40 # fixed problem statistics (drawn once, not re-rolled every time step)
41 z0_bar = np.array([2.0, -1.0, 3.0])
42 Sigma_0 = np.eye(3) * 0.6
43 Sigma_w = np.eye(3) * 0.05          # how much the target moves per step
44 x_sensors = np.random.uniform(-8, 8, (N, 3))   # drones stay put at these
       locations
45 Sigma_v = [np.eye(3) * s for s in np.random.uniform(0.15, 0.6, N)]  # each
       drone's own noise level
46 inv_Sigma_v = [np.linalg.inv(Sv) for Sv in Sigma_v]
47
48 # simulate the target actually wandering around (random walk)
49 z_true = np.zeros((T_max, 3))
50 z_true[0] = np.random.multivariate_normal(z0_bar, Sigma_0)
51 for t in range(1, T_max):
52     w = np.random.multivariate_normal(np.zeros(3), Sigma_w)
53     z_true[t] = z_true[t - 1] + w
54
55 # --- distributed gradient descent tracking ---
56 alpha = 0.04          # had to bring this down a lot - anything much bigger
       diverges
57 K_iters = 20           # inner iterations per time step before moving on
58 z_est = np.zeros((T_max, N, 3))
59 z_est[0] = np.tile(z0_bar, (N, 1))
60
61 for t in range(T_max):
```

```
    # every drone takes one noisy measurement at this time step
    y = np.zeros((N, 3))
    for i in range(N):
        v = np.random.multivariate_normal(np.zeros(3), Sigma_v[i])
        y[i] = x_sensors[i] - z_true[t] + v

    z_k = z_est[t - 1].copy() if t > 0 else z_est[0].copy()
    for _ in range(K_iters):
        consensus = W @ z_k
        grad = np.array([inv_Sigma_v[i] @ (z_k[i] - (x_sensors[i] - y[i])) for
    i in range(N)])
        z_k = consensus - alpha * grad
    z_est[t] = z_k

z_est_mean = z_est.mean(axis=1)
error = np.linalg.norm(z_est_mean - z_true, axis=1)
print(f"Mean steady-state ||e(t)|| over last 20 steps: {error[-20:].mean():.4f}
    ")

# --- plot 1: just the graph, to show it's connected ---
fig_g, ax_g = plt.subplots(figsize=(4, 4))
pos = nx.spring_layout(G, seed=1)
nx.draw(G, pos, ax=ax_g, with_labels=True, node_color="#ffd28a",
        node_size=420, font_weight="bold", edge_color="#555")
ax_g.set_title(f"Drone communication graph $\\mathcal{{G}}$ (N={N}, p={p_prob})
    ")
fig_g.tight_layout()
fig_g.savefig("figs/p2_graph.png", dpi=200)
plt.close(fig_g)

# --- plot 2: true path vs. what the swarm thinks is happening, plus the error
    ---
fig = plt.figure(figsize=(12, 5))
ax1 = fig.add_subplot(121, projection="3d")
ax1.plot(z_true[:, 0], z_true[:, 1], z_true[:, 2], "g-", lw=2, label="True
    intruder $z(t)$")
ax1.plot(z_est_mean[:, 0], z_est_mean[:, 1], z_est_mean[:, 2], "r--", lw=2,
    label="DGD estimate $\\hat{z}(t)$")
ax1.scatter(x_sensors[:, 0], x_sensors[:, 1], x_sensors[:, 2], c="b", marker="^
    ", s=60, label="Drones")
ax1.scatter(*z_true[0], c="k", marker="o", s=50)
ax1.set_xlabel("x"); ax1.set_ylabel("y"); ax1.set_zlabel("z")
ax1.set_title("Intruder tracking in $\\mathbb{R}^3$")
ax1.legend(fontsize=8)

ax2 = fig.add_subplot(122)
ax2.plot(range(T_max), error, "k-", lw=2)
ax2.set_xlabel("Time step $t$")
ax2.set_ylabel(r"Estimation error $\|e(t)\|$")
ax2.set_title("$\\|\\hat{z}(t)-z(t)\\|$ over time")
ax2.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("figs/p2_tracking.png", dpi=200)
plt.close(fig)

print("done - figures saved")
```

Problem 3
N agents on an undirected connected graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ must collaboratively execute $M > N$ tasks $\mathcal{T} = \{T_1, \dots, T_M\}$. Agent i has cost C_{ij} for task T_j (cost matrix $C \in \mathbb{R}^{N \times M}$) and capacity $b_i \in \mathbb{Z}_{\geq 0}$ with $\sum_i b_i = M$.

(1) Formulate task allocation as an optimization problem with the ADMM updates for every agent i. (Hint: binary program → NP-hard → use a convex relaxation.)

(2) Randomly fix $M > N$, generate a connected Erdos-Renyi graph, fix capacities b_i summing to M, and a random positive cost matrix C. Plot the graph and the ADMM task allocation. Re-run with a different C (same N, M, b) and explain the observed differences.

Solution:

(1) Formulation and ADMM updates. Let $X \in \{0,1\}^{N \times M}$ with $X_{ij} = 1$ iff agent i is assigned task j; this binary program is NP-hard for large M, N, so I relax it to $X_{ij} \in [0,1]$. Let $x_i, c_i \in \mathbb{R}^M$ be row i of X, C. The relaxed problem is

$$\min_{\{x_i\}} \sum_{i=1}^{N} c_i^\top x_i \quad \text{s.t.} \quad x_i \in \mathcal{X}_i := \{x_i : \mathbf{1}^\top x_i = b_i,\ 0 \leq x_i \leq 1\}\ \ \forall i, \qquad \sum_{i=1}^{N} x_i = \mathbf{1}_M,$$

i.e. each agent's local capacity constraint is private, while the "every task done exactly once" constraint is global – exactly the ADMM *sharing-problem* structure, which I rewrite as $\frac{1}{N}\sum_i x_i = \frac{1}{N}\mathbf{1}_M$.

$\mathcal{G}$ is only a communication graph, so there's no central node to compute that global average for everyone. I get around this by giving every agent its *own* scaled dual $u_i \in \mathbb{R}^M$ and having it estimate the network average purely from talking to its neighbours over $\mathcal{G}$, using R rounds of the doubly-stochastic averaging matrix W (the same idea as Problem 2). With penalty $\rho > 0$, the fully distributed ADMM updates I run at outer iteration k are:

$$x_i^{k+1} = \arg\min_{x_i \in \mathcal{X}_i}\ c_i^\top x_i + \frac{\rho}{2}\left\| x_i - x_i^k + \left(WX^k\right)_i - \tfrac{1}{N}\mathbf{1}_M + u_i^k \right\|_2^2 \quad \text{(local primal QP)} \tag{1}$$

$$\bar{x}_i^{k+1} = \left(W^R X^{k+1}\right)_i \quad (R\text{-round local average consensus}) \tag{2}$$

$$u_i^{k+1} = u_i^k + \bar{x}_i^{k+1} - \tfrac{1}{N}\mathbf{1}_M \quad \text{(local dual update)} \tag{3}$$

where X^k stacks all x_i^k and $(WX^k)_i$ is agent i's one-hop mixing of its neighbours' current iterates – I feed this straight into agent i's own QP so the primal step only ever needs information from its neighbours. As $R \to \infty$, $\bar{x}_i^{k+1} \to \frac{1}{N}\sum_j x_j^{k+1}$ for every i and (1)–(3) reduce to the standard centralized sharing-ADMM iteration; with finite R, this is the fully graph-respecting distributed version I actually simulate below.

(2) Simulation and observations. I used $N = 5$ agents, $M = 15$ tasks, capacities $b = (5, 3, 1, 5, 1)$ (randomly drawn, summing to 15), $\rho = 2$, 150 outer iterations, and $R = 8$ consensus rounds per iteration, with the local QPs solved via `cvxpy`/OSQP.

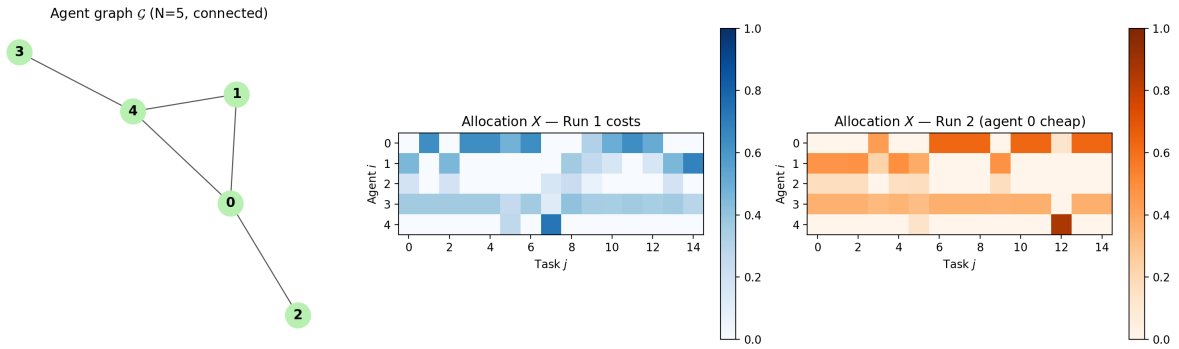


Figure 6: Left: connected agent graph $\mathcal{G}$ ($N = 5$). Middle: converged allocation X under the baseline random cost matrix C_1. Right: converged allocation under C_2, identical except agent 0's costs are scaled down by 4× (all else fixed: same graph, same b).

Observations. In both runs, each agent's row sums to exactly its capacity b_i – that's a hard local constraint, enforced exactly inside every local QP. Each task's column, on the other hand, only sums to 1 approximately (max deviation ≈ 0.03 for C_1 and ≈ 0.004 for C_2 after 150 iterations), since the global coverage constraint is only enforced asymptotically through the distributed averaging and dual feedback – a longer run or a larger ρ tightens this further. Comparing the two heatmaps, under C_1 agent 0 shares many tasks with agent 3 at middling fractional values, since the two are fairly evenly matched on cost. Once I make agent 0 four times cheaper (C_2), its allocation values on the tasks it wins go up and it visibly displaces the other agents on those columns, while agent 3 – its main competitor – shrinks back to what's left over. The allocation never becomes perfectly binary under either cost matrix, which is just the quadratic-penalty relaxation showing itself: getting a genuinely integral assignment out of this would need a separate rounding step, or a decentralized auction algorithm, on top of what ADMM gives here – exactly what the hint about NP-hardness was pointing at.

Python implementation

```
"""
Problem 3 - splitting M tasks across N agents using ADMM.

The catch is that standard sharing-ADMM assumes someone can see everyone's
variables at once to compute the average, which defeats the point of having
a communication graph in the first place. So instead of a central average,
each agent estimates the network average itself by gossiping with its
neighbors for a few rounds every iteration.
"""
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
import cvxpy as cp

np.random.seed(11)

# --- setup ---
N = 5    # agents
M = 15   # tasks - has to be more than N per the problem

# connected graph again, same approach as problem 2
while True:
    G = nx.erdos_renyi_graph(N, 0.5, seed=np.random.randint(0, 10_000))
    if nx.is_connected(G):
```

```
        break

A = nx.adjacency_matrix(G).toarray().astype(float)
deg = A.sum(axis=1)
max_deg = deg.max()
W = np.zeros((N, N))
for i in range(N):
    for j in range(N):
        if i != j and A[i, j] == 1:
            W[i, j] = 1.0 / (max_deg + 1)
    W[i, i] = 1.0 - W[i, :].sum()

# random capacities that add up to M - reroll if anyone ends up with 0
b = np.random.multinomial(M, np.ones(N) / N)
while (b == 0).any():
    b = np.random.multinomial(M, np.ones(N) / N)
print("Agent capacities b_i:", b, " sum =", b.sum())


def run_decentralized_admm(C, rho=2.0, iters=150, consensus_rounds=8):
    """
    Runs ADMM where nobody has a global view - each agent only ever talks to
    its neighbors. consensus_rounds sets how many gossip rounds happen per
    ADMM iteration to approximate the true network average.
    """
    X = np.zeros((N, M))
    U = np.zeros((N, M))
    target = np.ones(M) / N

    for _ in range(iters):
        # every agent solves its own small QP using only what it can see
    locally
        X_next = np.zeros((N, M))
        for i in range(N):
            x_var = cp.Variable(M)
            resid = x_var - X[i, :] + (W @ X)[i, :] - target + U[i, :]
            objective = cp.Minimize(C[i, :] @ x_var + (rho / 2) * cp.
    sum_squares(resid))
            constraints = [cp.sum(x_var) == b[i], x_var >= 0, x_var <= 1]
            cp.Problem(objective, constraints).solve(solver=cp.OSQP, verbose=
    False)
            X_next[i, :] = x_var.value
        X = X_next

        # a few rounds of gossiping so everyone's estimate of the average gets
    closer
        Xbar_local = X.copy()
        for _ in range(consensus_rounds):
            Xbar_local = W @ Xbar_local

        # dual update, using each agent's own (approximate) average
        U = U + Xbar_local - target

    return X


# run 1 - just some random costs to start with
C1 = np.random.uniform(1, 10, (N, M))
X1 = run_decentralized_admm(C1)

# run 2 - same setup, but make agent 0 way cheaper and see what shifts
C2 = np.random.uniform(1, 10, (N, M))
C2[0, :] *= 0.25
```

```
X2 = run_decentralized_admm(C2)

for name, X in [("Run 1 (baseline costs)", X1), ("Run 2 (agent 0 cheap)", X2)]:
    row_sums = X.sum(axis=1)
    col_sums = X.sum(axis=0)
    print(f"{name}: row sums (should equal b_i) = {np.round(row_sums, 2)}, "
          f"max |col_sum-1| = {np.max(np.abs(col_sums - 1)):.4f}")

# --- plot everything: the graph plus both allocation heatmaps side by side ---
fig = plt.figure(figsize=(15, 4.5))

ax0 = fig.add_subplot(131)
pos = nx.spring_layout(G, seed=2)
nx.draw(G, pos, ax=ax0, with_labels=True, node_color="#b7f0b0", node_size=500,
        font_weight="bold", edge_color="#555")
ax0.set_title(f"Agent graph $\\mathcal{{G}}$ (N={N}, connected)")

ax1 = fig.add_subplot(132)
im1 = ax1.matshow(np.round(X1, 2), cmap="Blues", vmin=0, vmax=1)
ax1.set_title("Allocation $X$ \u2014 Run 1 costs")
ax1.set_xlabel("Task $j$"); ax1.set_ylabel("Agent $i$")
ax1.xaxis.set_ticks_position("bottom")
fig.colorbar(im1, ax=ax1, fraction=0.046)

ax2 = fig.add_subplot(133)
im2 = ax2.matshow(np.round(X2, 2), cmap="Oranges", vmin=0, vmax=1)
ax2.set_title("Allocation $X$ \u2014 Run 2 (agent 0 cheap)")
ax2.set_xlabel("Task $j$"); ax2.set_ylabel("Agent $i$")
ax2.xaxis.set_ticks_position("bottom")
fig.colorbar(im2, ax=ax2, fraction=0.046)

fig.tight_layout()
fig.savefig("figs/p3_admm.png", dpi=200)
plt.close(fig)

print("figures saved")
print("\nRun 1 allocation (rounded):\n", np.round(X1, 2))
print("\nRun 2 allocation (rounded):\n", np.round(X2, 2))
```

Problem 4
Consider the LASSO problem

$$\min_x \left[\tfrac{1}{2}\|Ax-b\|_2^2 + \lambda\|x\|_1 \right].$$

Write the ADMM iteration updates, solving and writing the closed-form solution for the x and z updates.

Solution:

I introduce a splitting variable $z \in \mathbb{R}^n$ to decouple the smooth quadratic term from the non-smooth ℓ_1 term:

$$\min_{x,z} \ \tfrac{1}{2}\|Ax-b\|_2^2 + \lambda\|z\|_1 \quad \text{s.t.} \quad x - z = 0.$$

With scaled dual variable $u = y/\rho$, the scaled augmented Lagrangian is

$$\mathcal{L}_\rho(x,z,u) = \tfrac{1}{2}\|Ax-b\|_2^2 + \lambda\|z\|_1 + \tfrac{\rho}{2}\|x-z+u\|_2^2,$$

and ADMM alternates minimizing over x, then z, then takes a dual ascent step on u.

x-update. Minimizing $\mathcal{L}_\rho$ over x with z, u fixed is an unconstrained quadratic:

$$x^{k+1} = \arg\min_x \ \tfrac{1}{2}\|Ax-b\|_2^2 + \tfrac{\rho}{2}\|x - z^k + u^k\|_2^2.$$

Setting the gradient to zero, $A^\top(Ax-b) + \rho(x - z^k + u^k) = 0$, gives the closed form

$$\boxed{x^{k+1} = \left(A^\top A + \rho I\right)^{-1}\left(A^\top b + \rho\left(z^k - u^k\right)\right)}$$

I only need to compute $(A^\top A + \rho I)^{-1}$ (or its Cholesky/LU factors) once and reuse it every iteration, since it doesn't depend on k.

z-update. Minimizing over z isolates the proximal operator of the ℓ_1 norm:

$$z^{k+1} = \arg\min_z \ \lambda\|z\|_1 + \tfrac{\rho}{2}\|x^{k+1} - z + u^k\|_2^2 = \operatorname{prox}_{(\lambda/\rho)\|\cdot\|_1}\left(x^{k+1} + u^k\right).$$

This proximal operator has the classical element-wise soft-thresholding closed form, $S_\kappa(a) = \operatorname{sign}(a)\max(0, |a| - \kappa)$, so

$$\boxed{z^{k+1} = S_{\lambda/\rho}\left(x^{k+1} + u^k\right)}, \qquad \left(z^{k+1}\right)_m = \operatorname{sign}\left((x^{k+1}+u^k)_m\right)\max\left(0,\ \left|(x^{k+1}+u^k)_m\right| - \tfrac{\lambda}{\rho}\right).$$

u-update (dual ascent).

$$\boxed{u^{k+1} = u^k + x^{k+1} - z^{k+1}}$$

Putting these three closed-form updates together gives me the full ADMM iteration for LASSO: an x-step that's a single linear solve, a z-step that's a cheap elementwise shrinkage, and a u-step that's just a vector addition. That means each ADMM iteration only costs $O(n^3)$ once, for the cached factorization, and $O(n)$ after that – which is exactly why ADMM scales so much better to large sparse-regression problems than solving the LASSO directly as one big QP.